과제 5 요구사항 요약

커피숍 주문 시스템

다수 서버 환경에서도 안정적으로 동작하는 커피숍 주문 시스템을 설계하고 구현하는 과제입니다.

- 이 문서는 과제의 전체 흐름과 주요 요구사항을 정리한 **요약 자료**입니다.
- 이 과제는 기존 과제들과 달리 **요구사항이 명확하게 정의되어 있지 않습니다.** API 형식과 인프라는 직접 정합니다.
- 세부 요구사항은 저장소의 `README.md`를 기준으로 확인해 주세요.

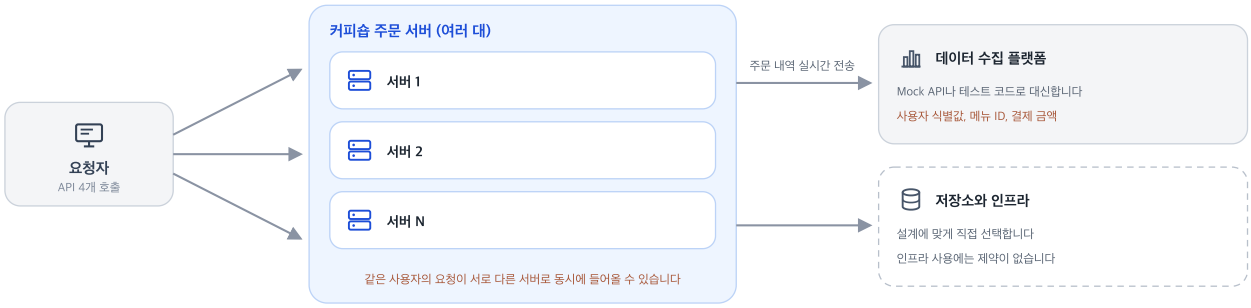
4개
필수 API
메뉴 조회, 포인트 충전, 주문과 결제, 인기 메뉴 조회

4가지
도전 요구사항
다수 인스턴스, 동시성, 데이터 일관성, 테스트

15개
자가 진단 항목
완성도, 이해도, 우수성으로 나누어 점검합니다

제약 없음
인프라 사용
DB, 캐시 등은 설계에 맞게 직접 선택합니다

전체 구조



파란색은 직접 구현할 서버, 회색은 Mock으로 대신하는 외부 시스템, 점선은 직접 선택하는 부분입니다.

과제 목표와 직접 정하는 것

과제 목표

- 필수 API 4개를 요구사항에 맞게 구현합니다.
- 서버가 **여러 대**여도 기능에 문제가 없도록 설계합니다.
- 동시성, 데이터 일관성, 확장성 관점에서 고른 방법과 **그 이유**를 README에 설명합니다.

직접 정하는 것

- API 경로와 HTTP 메서드, 요청과 응답 형식
- 에러 응답과 에러 코드
- DB, 캐시 등 사용할 인프라
- 데이터 수집 플랫폼을 대신할 Mock 방식

필수 API 4개

API	요구사항
1. 커피 메뉴 목록 조회	커피 정보(메뉴 ID, 이름, 가격)를 조회합니다.
2. 포인트 충전	사용자 식별값과 충전 금액을 입력받아 포인트를 충전합니다. 결제는 포인트로만 할 수 있습니다. (1원 = 1P)
3. 커피 주문과 결제	사용자 식별값과 메뉴 ID를 입력받아 주문하고, 충전한 포인트에서 주문 금액을 차감합니다. 주문 내역(사용자 식별값, 메뉴 ID, 결제 금액)을 데이터 수집 플랫폼으로 **실시간** 전송합니다.
4. 인기 메뉴 목록 조회	최근 7일 동안 인기 있는 메뉴 3개를 조회합니다. 메뉴별 주문 횟수가 **정확해야** 합니다.

도전 요구사항 4가지

필수 API를 설계할 때부터 아래 4가지를 함께 고려해 주세요.

도전 1

다수 서버, 다수 인스턴스

애플리케이션이 여러 서버에서 여러 인스턴스로 동작해도 기능에 문제가 없도록 설계합니다.

도전 2

동시성

동시에 요청이 들어오는 상황을 고려해 구현합니다.

도전 3

데이터 일관성

트랜잭션 관점에서 데이터 일관성을 고려해 구현합니다.

도전 4

테스트

각 기능과 제약사항에 대한 테스트를 작성합니다.

README에 반드시 포함할 내용

문제 해결 전략

- 설계 내용 (ERD, API 명세서)
- 설계 의도
- 선택한 문제 해결 전략과, 이 전략을 고르기 위해 분석한 내용
- 기술을 선택한 이유

추가 조건

- 인프라 사용에는 제약이 없습니다.
- HTTP 메서드(`GET` , `POST` , `PUT` , `DELETE`)는 자유롭게 선택합니다.
- 에러 응답과 에러 코드는 자유롭게 정의합니다.

자가 진단 체크리스트 15개

완성도 8개

- 필수: API 4개를 요구사항에 맞게 구현
- 도전: 다수 서버, 다수 인스턴스에서 정상 동작
- 도전: 동시 요청에도 정합성 유지
- 도전: 트랜잭션 관점의 데이터 일관성
- 도전: 기능과 제약사항별 단위 테스트

이해도 4개

- 필수: TIL에 트러블슈팅 과정 기록
- 도전: 디렉토리와 파일 분리
- 도전: 직관적인 변수명과 코드
- 도전: 주석으로 코드 설명

우수성 3개

- 필수: README로 과제 소개
- 도전: 특수 상황 예외 처리 2가지 이상
- 도전: 커밋 컨벤션을 지킨 커밋 10회 이상

권장 진행 순서

1. **이 문서로 과제의 전체 흐름을 확인합니다.**
2. **README.md를 처음부터 끝까지 읽습니다.**
   필수 요구사항과 도전 요구사항을 함께 확인해 주세요.
3. **설계 내용을 정리합니다.**
   ERD, API 명세서, 문제 해결 전략과 기술 선택 이유를 먼저 정리해 주세요.
4. **튜터와 설계 점검을 진행합니다.**
   설계를 정리한 뒤 바로 구현하지 말고, 설계 점검을 먼저 받는 것을 권장합니다.
5. **`starter-project/` 에서 구현과 테스트를 진행합니다.**
   기능을 하나 구현할 때마다 커밋하는 것을 권장합니다.
6. **README를 완성하고 자가 진단 체크리스트로 점검합니다.**

정해진 정답이 있는 과제가 아닙니다. 왜 이런 구조로 설계했는지 본인의 논리로 설명해 주세요.
세부 요구사항은 저장소의 **README.md**를 기준으로 확인해 주세요.